

Introduction to Machine Learning - Neural Networks Coursework

Vlad Coroian (vc620), Angelo Giacco (asg20),
Adi Boroica (ab6020), Nicolas Anguita (na920)

December 5, 2022

1 Model Description

1.1 Pytorch

We decided to use Pytorch instead of our Part 1 model for various reasons:

1. Pytorch was developed by hundreds of engineers at Meta, and the main logic is implemented in Python, C++ and CUDA. This makes it much more efficient and consequently faster.
2. Pytorch contains the state of the art techniques in Machine Learning. This means we can improve our model with the latest developments in the field by calling previously written functions. An example of this is the Hyperband algorithm in Hyperparameter Tuning.
3. Since Pytorch is so common in the Machine Learning community, there is a lot of guidance online which has proven helpful throughout the coursework.

Something else that we considered was how to initialise the weights of the neural network. We considered various alternatives including Xavier, Normalised Xavier and He optimisation and concluded that the default Pytorch implementation produces great accuracy while maintaining simplicity. This implementation samples the weights from a normal distribution.

1.2 Preprocessor

We debated several steps of the preprocessor method:

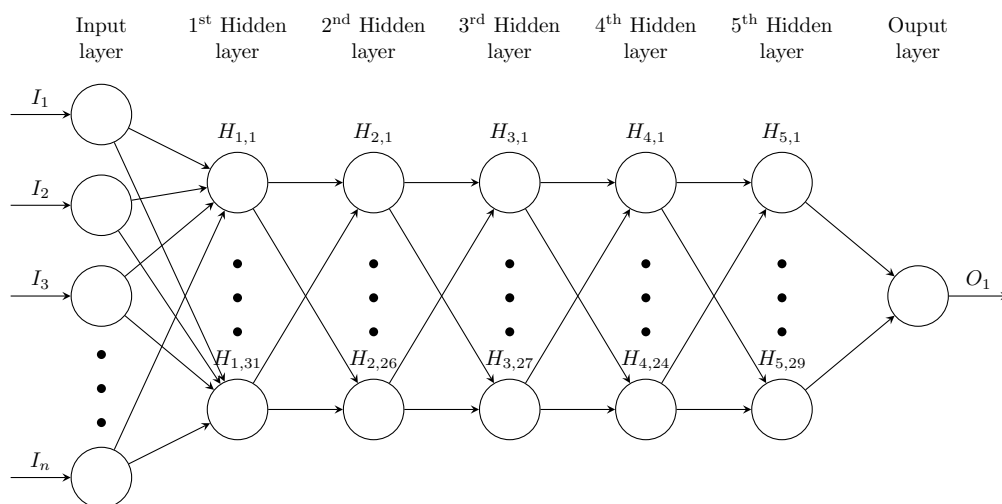
- **Normalisation:** there are two main ways to normalise columns - MinMax and StandardScalar. We ultimately decided to go with MinMax as it is guaranteed to return a number between 0 and 1. This makes the model easier to train and completely eliminates the potential of overflow. In contrast you can still have large values with StandardScalar (although with a small probability)
- **Missing values:** an important decision was whether to remove rows with missing values or replace them with a reasonable guess. We have decided to follow the following strategy to deal with missing values: if the row had more than half of the values missing that we did not include it in the training, but otherwise filled in the missing data, as we considered that this data point can still give valuable insight. For numerical columns, we decided to replace missing values with the median as opposed to the mean. The key difference is that the median is less affected by outliers so is a more robust metric. Missing values in categorical columns were replaced with the most common value (mode).
- **One Hot Encoding:** there are two main ways to deal with categorical data in the preprocessing step: one hot encoding and ordinal encoding. One hot encoding is generally preferred when preprocessing data for neural nets as cardinal encoding makes it seem as if there is a ranking between inputs, when there isn't necessarily.

1.3 Neural Network

- **Activations:** We decided to only have ReLU activations since the other options were not as suitable (Sigmoid and Arctan are only use for classification, not regression). Through experiments we have decided that it is best to use ReLU in all layers.
- **Width:** the number of neurons in each layer of the network was decided by testing random values from a uniformly uniform distribution [15, 35]. More on this on hyperparameter tuning.
- **Number of layers:** the number of layers of the network was determined similarly, by sampling from a uniform distribution and picking the amount which maximises the score.

1.4 Best performing model

Our best performing model achieved a RMSE of around 54000. Examining this model we observe that this has 7 linear layers (with a ReLU layer between these, 1 input layer, 1 output layer and 5 hidden layers). The number of neurons on each hidden layer are (in order): 31, 26, 27, 24, 29. We also observe that this model has been trained with batch size of 100, a learning rate of 0.0999, and 893 epochs.



2 Model Evaluation

To measure the accuracy of our predictions, we decided to go with MSE . It can be shown that:

$$MSE_{\theta}(T) = Var_{\theta}(T) + Bias_{\theta}(T, \theta)^2$$

This shows that by trying to reduce the MSE we are reducing both the variance and the Bias of our model predictions. This is preferred compared to using just the Bias as an accuracy metric. In addition, MSE penalises large errors more heavily than small errors. This makes it an ideal metric for our application, as we'd rather have lots of small errors when predicting house prices than frequently not having an error but occasionally having a huge error.

In order to evaluate the model properly, we did held out testing. We split the dataset into three splits: 80% was used for training the neural network, 10% was used for validation and 10% was used for testing the network after the hyperparameter tuning. Once hyperparameter tuning was complete (as laid out below) the best performing hyperparameters were then tested on the held out test set.

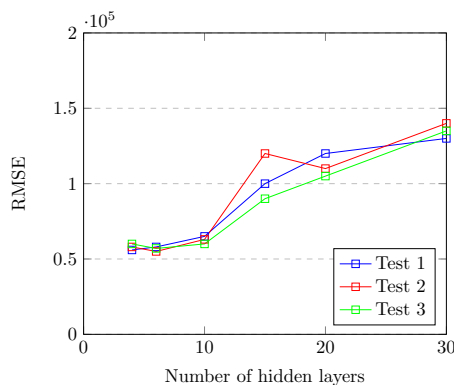
3 Hyperparameter tuning

The hyperparameters we have decided to tune were the number of layers of the network, the width of every layer, the number of epochs and the batch size. We have also considered trying other types of optimisers and activation functions, but eventually decided to keep those fixed as the values we chose consistently got better results when we did our initial experiments. As it is very computationally expensive to fit the training data for the model we have decided to first carry out a few experiments with parameters in a certain range and then decide whether it's worth carrying out a hyperparameter search in that range (e.g. we've tried setting 15 layers of neurons, but got very bad scores for models trained in this way, so we've decided to limit our search to models with 4 to 10 layers).

We also considered running a genetic algorithm to try and find the ideal hyperparameters, however we concluded that this was infeasible due to the computation required by the amount of generations needed to find good, novel solutions. Thus, we opted for our intuitive approach instead.

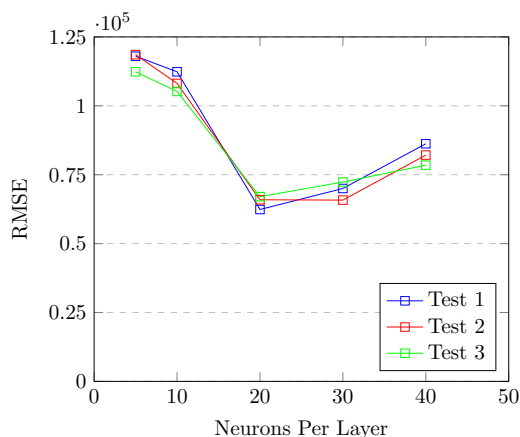
3.1 Number of Hidden Layers

Based on our trials, we decided to pick the number of hidden layers a random between 5 and 10. A large number of layers increase the computation time and can lead to overfitting.



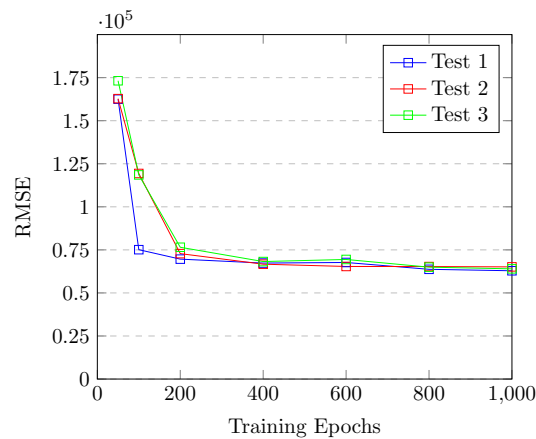
3.2 Number of neurons per layer

Based on our trials, we've decided that every layer would contain a random number of neurons between 15 and 35. A large number of neurons increase the computation time and can lead to overfitting. On the other hand, a small number of neurons can lead to underfitting.



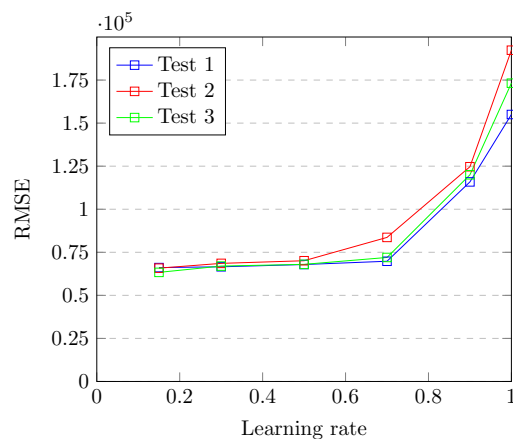
3.3 Training epochs

Based on our trials, we decided to pick the number of training epochs by choosing a random number between 700 and 1000.



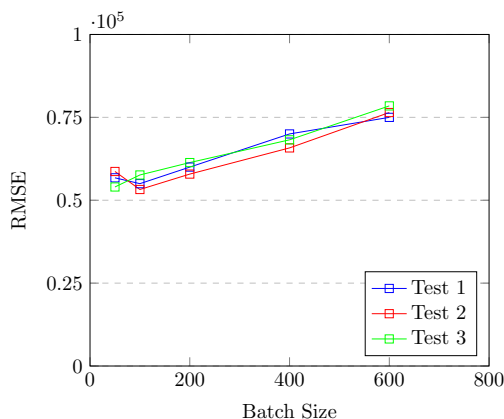
3.4 Learning rate

Based on our trials, we decided to pick the learning rate by choosing a random number between 0.05 and 0.2. This helped us get to a model that does not result in overflow, and it also got to convergence in a reasonable number of epoch (and in reasonable time).



3.5 Batch size

We found that the models trained a lot slower when using very small batch sizes, for example of 20 or 50. In addition, their performance was not as good as slightly larger batch sizes. Much larger batch sizes (> 300) performed significantly worse. Based on our trials, we decided to pick the size by choosing a random number between 100 and 200.



3.6 Activation layers

We decided to have only ReLU activation layers, since:

- SigmoidLayer and TanhLayer are used for classifying into two classes.
- SoftmaxLayer is used for predicting a probability distribution.
- Having an IdentityLayer would not make sense since we're using only linear layers. This means that the composition of LinearLayer $\rightarrow$ IdentityLayer $\rightarrow$ LinearLayer would be equivalent to a LinearLayer.

Moreover, the last layer should always be ReLU, since the price of a house can not be negative.

4 Final evaluation of the model

A randomly initialized model usually gives around 75.000–85.000 for RMSE, and after hyperparameter tuning we reduce this consistently to about 55.000 – 60.000, with our best performing model getting around 54.000. In section 1.4 we have also described the structure of this model.